

บทที่ 3

การทำงานกับไฟล์

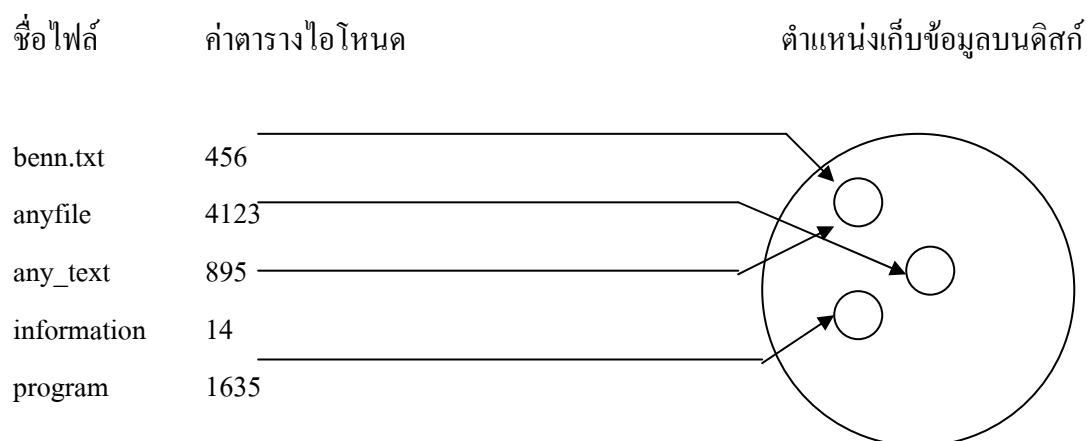
3.1 โครงสร้างของไฟล์บนระบบยูนิกซ์

ไฟล์ในระบบยูนิกซ์เป็นสิ่งที่มีความสำคัญ เนื่องจากการทำงานกันอย่างใกล้ชิดระหว่างระบบปฏิบัติการ, อุปกรณ์และ ระบบไฟล์ และที่สำคัญยูนิกซ์มองทุกสิ่งทุกอย่างที่เป็นไฟล์ โดยโปรแกรมสามารถใช้งานดิสก์ พอร์ตอนุกรม พรินเตอร์ หรืออุปกรณ์อื่น ๆ เสมือนหนึ่งว่าเป็นไฟล์ โดยจะครอบคลุมการใช้งานฟังก์ชันที่ใช้งานกันบ่อย 5 ฟังก์ชันคือ open, close, read, write และ ioctl

3.2 ไดรเรกทอรีที่สำคัญบนยูนิกซ์

สำหรับไดเรกทอรีเป็นไฟล์ชนิดหนึ่ง การที่จะลบไดเรกทอรีโดยใช้คำสั่ง rmdir จะต้องไม่มีข้อมูลในไดเรกทอรีนั้น (ถึงแม้ว่าจะเป็น root ก็ตาม) นอกจากนี้ยังมีคำสั่ง opendir/readdir เพื่อสร้างไดเรกทอรี

ไดเรกทอรีเป็นไฟล์เฉพาะรูปแบบหนึ่ง เก็บค่าตารางไอโฟนดต่อรายชื่อไฟล์ที่อยู่ในไดเรกทอรี รายชื่อของไฟล์ที่อยู่ในไดเรกทอรีก็คือ ลิงค์ (link) ไปยังรายชื่อไฟล์บนไอโฟนด, การลบไฟล์คือการลบลิงค์ ถ้าไม่มีไฟล์ในไดเรกทอรีก็คือ ไม่มีลิงค์ในตารางไอโฟนด, พื้นที่ในตารางจะว่าง ไฟล์อื่นสามารถเอาไปใช้งานได้



รูปที่ 3 – 1 ค่าตารางไอโฟนดและตำแหน่งที่เก็บบนแผ่นดิสก์

ที่ตารางไอโฟนด จะเก็บข้อมูลส่วนอื่นของไฟล์ เช่น วันที่สร้างหรือแก้ไขไฟล์ เพอร์มิสชันในการแอ็กเซสไฟล์/ไดเรกทอรี ความยาวของไฟล์ หรือตำแหน่งข้อมูลบนดิสก์

ไฟล์จะถูกเก็บไว้ในไดเรกทอรี ในรูปแบบโครงสร้างของไฟล์แบบต้นไม้ สำหรับไฟล์ผู้ใช้งานคนนั้น เช่น richie จะมีโฮมไดเรกทอรี /home/richie ภายในโฮมไดเรกทอรีอาจแบ่งเป็นไดเรกทอรีเก็บ

อีเมลล์ จดหมายติดต่อบริษัท ทูลที่ใช้ทำงานต่าง ๆ สำหรับโฮมไดรากทอรี ก็เป็นซัพไดรากทอรีย่อยในไดรากทอรีที่อยู่เหนือขึ้นไปอีกต่อหนึ่ง, ในกรณีของลินุกซ์คือ /home

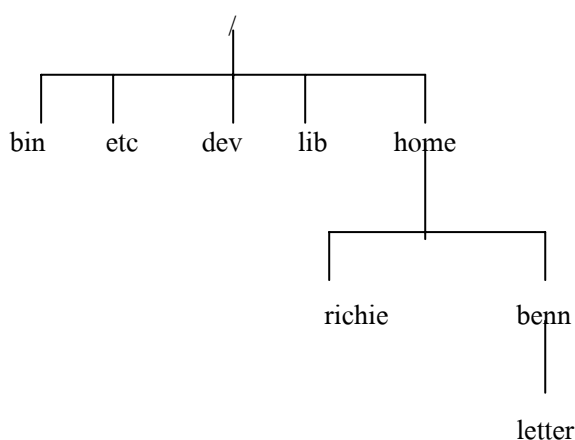
ภายใต้ root, /, นอกจากนี้จะมี /home แล้วยังซัพไดรากทอรีอื่นอีก เช่น

/bin เก็บโปรแกรมของระบบ

/etc เก็บคอนฟิกูเลชันของระบบ

/lib เก็บไฟล์ไลบรารี

/dev สำหรับไฟล์ที่ใช้แทน อุปกรณ์รวมทั้งสนับสนุนการติดต่อกับอุปกรณ์เก็บไว้ที่ไดรากทอรี



รูปที่ 3 – 2 โครงสร้างของไฟล์แบบต้นไม้

3.3 ไฟล์ของอุปกรณ์

ยูนิกซ์จะมองอุปกรณ์เป็นไฟล์ เช่น ตัวอย่างต่อไปนี้เป็น การ mount ซีดีรอม ให้เป็นไฟล์

```
$ mount -t iso9660 /dev/hdc /mnt/cdrom
```

```
$ cd /mnt/cdrom
```

จะสามารถเอ็กเซสซีดีรอมได้โดยผ่านไดรากทอรี /mnt/cdrom สำหรับไฟล์อุปกรณ์ชนิดอื่น ๆ มีดังนี้

/dev/console

เป็นคอนโซลของระบบ ข้อความ error และผลการทำงานต่าง ๆ จะถูกส่งมายังไฟล์นี้ ในยูนิกซ์จะต้องมีเทอร์มินอลที่ได้รับการกำหนดให้เป็นคอนโซลของระบบ อย่างน้อย 1 เทอร์มินอล ส่วนใหญ่จะเป็นคอนโซลที่กำลังแอ็กทีฟ หรือเป็นวินโดวส์ที่กำลังแอ็กทีฟ (ถ้ารันในระบบ X)

/dev/tty

เปรียบเสมือนไฟล์ที่ใช้ติดต่อกับเทอร์มินอลต่าง ๆ ของโปรเซส เช่น คีย์บอร์ด จอภาพเอาต์พุต หรือวินโดวส์ของระบบ X สำหรับ /dev/console มีได้เพียงไฟล์เดียวส่วน /dev/tty มีหลายไฟล์ขึ้นกับอุปกรณ์ที่ใช้งาน

/dev/null

เป็น null device, ข้อมูลที่ถูกเขียนใส่ดีไวซ์ไฟล์นี้ จะถูกละทิ้ง(เหมือนลบข้อมูล นั่นเอง), เมื่อใช้คำสั่ง cp คัดลอกข้อมูลจากไฟล์ /dev/null จะได้ไฟล์เปล่าที่ไม่มีข้อมูลอะไร เช่นการใช้คำสั่งต่อไปนี้

```
$ echo send this message to bin > /dev/null
```

```
$ cp /dev/null empty_file
```

สำหรับอุปกรณ์อื่น ที่พบใน /dev เช่น ฮาร์ดดิสก์หรือฟลอปปีดิสก์ พอร์ตอนุกรมหรือพอร์ตขนาน เทปไดรฟ์, ซีดีรอม, การ์ดเสียง หรืออุปกรณ์ภายในของระบบอื่น ๆ จะแสดงสถานะภายในของอุปกรณ์นั้น สำหรับลินุกซ์ การจะแก้ไขไฟล์พวกนี้จะต้องเป็นซูเปอร์ยูสเซอร์(superuser) เท่านั้น

ชื่อของดีไวซ์ไฟล์ อาจจะแตกต่างกันไปตามระบบปฏิบัติการที่ใช้งาน

3.4 เรียกใช้งานฟังก์ชันระดับต่ำ

เราสามารถจะแอ็กเซสไฟล์ หรือควบคุมอุปกรณ์โดยใช้ฟังก์ชันระดับต่ำ (System calls)

ซึ่งจะมีในระบบยูนิกซ์พร้อมอยู่แล้ว ฟังก์ชันเหล่านี้จะได้รับบริการโดยตรงจากส่วนกลางของระบบปฏิบัติการ เรียกว่า เคอร์เนล(Kernel) ซึ่งเป็นกลุ่มของดีไวซ์ไดรเวอร์ ประกอบด้วยการอินเทอร์เฟซกับฮาร์ดแวร์ในระดับต่ำ เช่น สำหรับ เทปไดรฟ์ จะประกอบด้วยการเริ่มรันเทปไดรฟ์, การหมุนไปข้างหน้าหรือกลับหลัง การอ่านหรือการเขียนเทป นอกจากนี้ยังต้องใช้ข้อมูลบางอย่าง เช่น ขนาดของบล็อก รวมทั้งความจำเพาะต่ออุปกรณ์ เช่น เทป เป็นสื่อเก็บข้อมูลแบบ ซีควเอนเชียล แอ็กเซส (Sequentail Access) ระบบไม่สามารถจะแอ็กเซสข้อมูลโดยตรงได้

หรือฟังก์ชันระดับต่ำของฮาร์ดดิสก์ ก็จะสามารถเขียนข้อมูลที่ตำแหน่งใดของพื้นที่ดิสก์ก็ได้ เนื่องจากเป็นแรนดอมแอ็กเซสดีไวซ์(Random Access device)

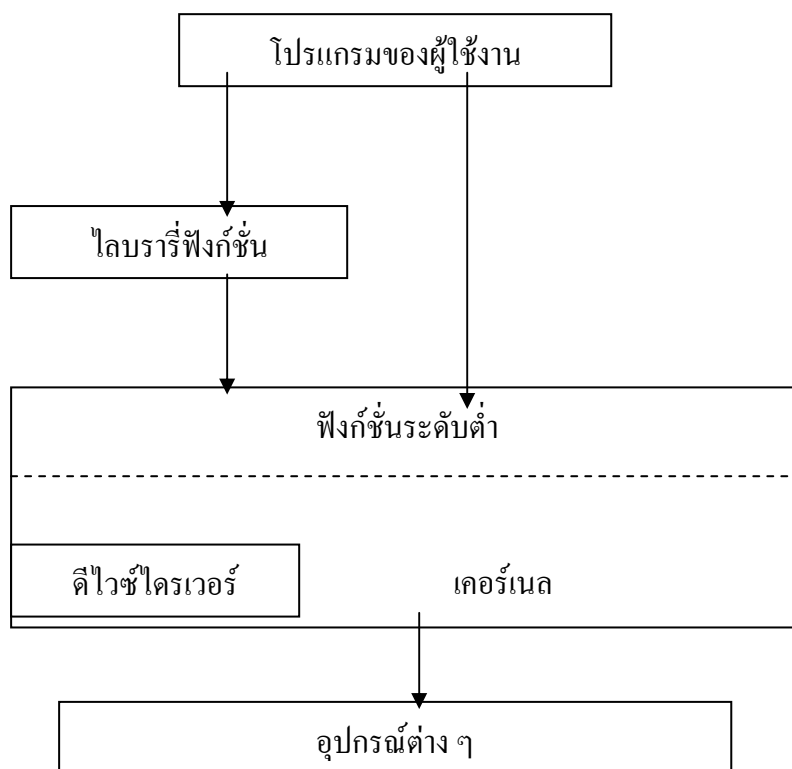
การควบคุมการทำงานเฉพาะอย่างของอุปกรณ์เหล่านี้ ทำได้โดยฟังก์ชันระดับต่ำ ioctl ดังนั้นการใช้งานจะแตกต่างกันไปตามแต่ละอุปกรณ์ เช่น การหมุนเทปเพื่อหาตำแหน่งข้อมูลหรือกำหนดโฟลคอนโทรล(flow control) สำหรับพอร์ตอนุกรม เป็นต้น

สำหรับดีไวซ์ไฟล์ใน /dev ส่วนใหญ่แล้ว จะใช้งานฟังก์ชันในลักษณะเดียวกันเช่น การแอ็กเซสไฟล์ (open) ไม่ว่าจะเป็น ดิสก์ เทอร์มินอล พรินเตอร์หรือเทปไดรฟ์

ฟังก์ชันระดับต่ำที่เป็นมาตรฐาน ในการแอ็กเซสดีไวซ์ไดรเวอร์(เคอร์เนล) มีดังนี้

open	เปิดไฟล์หรือการเริ่มการติดต่อกับอุปกรณ์
read	อ่านข้อมูลจากไฟล์หรืออุปกรณ์

write	เขียนข้อมูลลงไฟล์หรืออุปกรณ์
close	ปิดไฟล์หรืออุปกรณ์
ioctl	ฟังก์ชันควบคุมการทำงานเฉพาะอย่างสำหรับอุปกรณ์



รูปที่ 3 – 3 ฟังก์ชันการทำงานระดับต่ำ

3.5 ฟังก์ชันการแอ็กเซสไฟล์ในระดับต่ำ

โปรแกรมที่กำลังรันอยู่จะมีโปรเซสการทำงานของตัวเอง โปรเซสเหล่านี้จะทำงานร่วมกับไฟล์หรืออุปกรณ์ โดยใช้หมายเลขแทนไฟล์ (file descriptor) โดยปกติจะมีหมายเลขของ file descriptor อยู่ 3 แบบที่เปิดใช้งานอยู่แล้ว คือ

- 0 อุปกรณ์อินพุตมาตรฐาน
- 1 อุปกรณ์เอาต์พุตมาตรฐาน
- 2 อุปกรณ์แสดงerror

นอกจากนี้ยังสามารถทำงานร่วมกับไฟล์หรืออุปกรณ์อื่น ที่มี file descriptor โดยใช้คำสั่ง open เพื่อเริ่มต้น(initialize) การทำงานของ file descriptor และสามารถจะใช้คำสั่ง write หรือ read เพื่อเขียนหรืออ่านข้อมูลจากไฟล์ได้

write

```
#include <unistd.h>
```

```
size_t write(int fildes, const void *buf, size_t nbytes);
```

ฟังก์ชัน write จะนำข้อมูลจำนวน nbytes จากบัฟเฟอร์ buf ไปเขียนลงในไฟล์ที่มี file descriptor fildes โดยจะส่งค่ากลับเป็นจำนวนไบต์ที่เขียนจริง ๆ (ซึ่งอาจจะน้อยกว่า nbytes ก็ได้) ถ้าส่งค่ากลับเป็น 0 แสดงว่าถึงจุดสิ้นสุดของไฟล์ ถ้าส่งค่ากลับเป็น -1 แสดงว่ามี error เกิดขึ้นในการทำงานของ write

read

```
#include <unistd.h>
```

```
size_t read(int fildes, void *buf, size_t nbytes);
```

จะอ่านข้อมูล nbytes จากไฟล์ที่มี file descriptor fildes เก็บไว้ในบัฟเฟอร์ buf จะส่งค่ามาเป็นจำนวนที่อ่านได้จริง ถ้าส่งค่ากลับมาเป็น 0 หมายถึงไม่มีการอ่านข้อมูล ถ้ามี error เกิดขึ้นจะส่งค่ากลับเป็น -1

open

```
#include <fcntl.h>
```

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
int open(const char *path, int oflags);
```

```
int open(const char *path, int oflags, mode_t mode);
```

ในมาตรฐาน POSIX (รวมทั้งลินุกซ์) อาจจะไม่ต้องรวม types.h และ stat.h เข้าไปด้วยแต่ในระบบยูนิกซ์ บางระบบต้องรวมเข้าไปด้วย

คำสั่ง open จะสร้าง path ใหม่ในการเข้าถึงไฟล์(หรืออุปกรณ์) ถ้าทำได้สำเร็จจะส่งค่ากลับมาเป็น file descriptor ที่สามารถใช้ได้ในฟังก์ชัน read, write ได้, file descriptor นี้ จะแตกต่างกันไปแต่ละโปรเซสไม่สามารถใช้ร่วมกันได้ การทำงานของแต่ละ file descriptor จะต่อเนื่องกันไปเรื่อย เช่น จะอ่านข้อมูลจากจุดที่แล้ว(หน่วยความจำ) หรือจะเขียนข้อมูลต่อจากจุดที่แล้ว หลังจากหยุดทำงาน แต่ละ file descriptor จะมีออฟเซตเป็นของตัวเอง

path จะกำหนดไฟล์ที่เปิดหรือสร้างขึ้นมาใหม่

oflags จะกำหนดเพอร์มิชชันการเข้าถึงเมื่อเปิดไฟล์ โดยจะต้องใช้งานโหมดใดโหมดหนึ่งต่อไปนี้

โหมด	คำอธิบาย
O_RDONLY	อ่านได้เท่านั้น
O_WRONLY	สามารถแก้ไขไฟล์ได้
O_RDWR	อ่านและแก้ไขไฟล์ได้

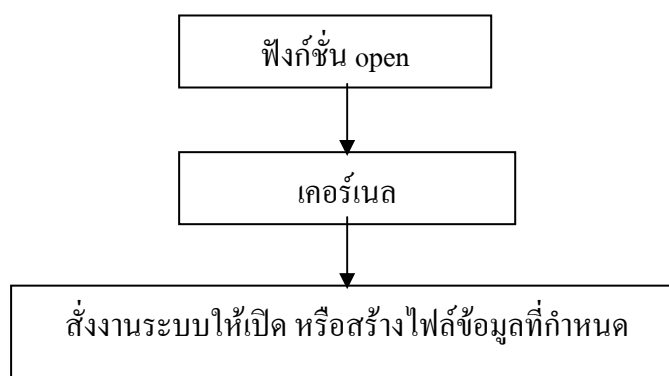
ตาราง 3 – 1 โหมดการทำงาน

นอกจากนี้ยังสามารถใช้งานออฟชั่นข้างต้น OR(bit wise) กับออฟชั่นต่อไปนี้

O_APPEND	เขียนข้อมูลต่อท้ายไฟล์
O_TRUNC	กำหนดความยาวของไฟล์เป็น 0 (ลบข้อมูลถ้ามีอยู่)
O_CREAT	สร้างไฟล์ ถ้าไม่มีไฟล์นั้น โดยใช้เพอร์มิสชันในการแอ็กเซสจากข้อมูลในหน้าถัดไป แต่ถ้ามีไฟล์อยู่แล้วจะเปิดไฟล์
O_EXCL	จะใช้ร่วมกับ O_CREAT ในการป้องกันไม่ให้ระบบสร้างไฟล์ดังกล่าวในเวลาเดียวกัน กับโปรแกรมอื่น ในกรณีที่รันในระบบ multitasking ในกรณีมีไฟล์อยู่แล้วจะส่งค่ากลับเป็น -1

ตารางที่ 3 – 2 ออฟชั่นเพิ่มเติม

ถ้าทำงานสำเร็จฟังก์ชัน open จะส่งหมายเลขไฟล์ที่ใช้งานใหม่กลับ(file descriptor) หรือส่ง -1 ถ้าทำงานไม่สำเร็จ และจะสามารถระบุสาเหตุที่ทำงานไม่สำเร็จไว้ที่ตัวแปร errno ค่า file descriptor จะเป็นค่าต่ำสุดที่ไม่มีการใช้งาน



รูปที่ 3 – 4 ขั้นตอนการทำงานของฟังก์ชัน open

mode เมื่อมีการสร้างไฟล์ใหม่โดยใช้งาน O_CREAT จะต้องกำหนดเพอร์มิสชันเริ่มต้นในการแอ็กเซสไฟล์โดยใช้ OR(bits wise) Operator กับแฟลกที่กำหนดไว้ในไฟล์ sys/types.h ดังต่อไปนี้

S_IRUSR	กำหนดให้ผู้ใช้เป็นเจ้าของไฟล์สามารถอ่านข้อมูลได้
S_IWUSR	กำหนดให้ผู้ใช้เป็นเจ้าของไฟล์สามารถแก้ไขข้อมูลได้

S_IXUSR	กำหนดให้ผู้เป็นเจ้าของไฟล์สามารถจะรันไฟล์ได้เท่านั้น
S_IRGRP	กำหนดให้กลุ่มที่เป็นเจ้าของไฟล์สามารถอ่านข้อมูลได้
S_IWGRP	กำหนดให้กลุ่มที่เป็นเจ้าของไฟล์สามารถแก้ไขข้อมูลได้
S_IXGRP	กำหนดให้กลุ่มที่เป็นเจ้าของไฟล์สามารถจะรันไฟล์ได้
S_IROTH	ผู้ใช้งานคนอื่นนอกเหนือจาก 2 กลุ่มแรกสามารถอ่านข้อมูลได้
S_IWOTH	ผู้ใช้งานคนอื่นนอกเหนือจาก 2 กลุ่มแรกสามารถแก้ไขข้อมูลได้
S_IXOTH	ผู้ใช้งานคนอื่นนอกเหนือจาก 2 กลุ่มแรกสามารถรันไฟล์ข้อมูลได้

นอกจากไฟล์เพอร์มิสชันแล้วจะกำหนดโดยคำสั่งข้างต้นแล้ว อีกคำสั่งหนึ่งก็คือ umask ซึ่งเป็นตัวแปรเชลล์ กำหนดระดับการห้ามให้กับเพอร์มิสชันของไฟล์เป็นหลักไว้ เมื่อมีการสร้างไฟล์ขึ้นมา

umask

เป็นตัวแปรเชลล์เก็บเพอร์มิสชันการแอ็กเซสไฟล์ จะใช้งานเมื่อมีการสร้างไฟล์และสามารถที่จะแก้ไขค่าในตัวแปร umask โดยจะประกอบด้วยตัวเลข 3 หลักในฐานะ 8 กำหนดการไม่อนุญาตสำหรับผู้ใช้งาน(user) กลุ่มผู้ใช้งาน (group) หรือผู้ใช้งานอื่น (others) ตามลำดับตามตารางต่อไปนี้

หลัก	ค่า	คำอธิบาย	เพอร์มิสชัน
1	0	อนุญาตให้เจ้าของมีการแอ็กเซสได้เต็มที่	rwX
	4	ไม่อนุญาตให้เจ้าของอ่านไฟล์ได้	-wX
	2	ไม่อนุญาตให้เจ้าของแก้ไขไฟล์ได้	r-X
	1	ไม่อนุญาตให้เจ้าของรันไฟล์ได้	rw-
2	0	อนุญาตให้กลุ่มผู้ใช้งานมีการแอ็กเซสได้เต็มที่	rwX
	4	ไม่อนุญาตกลุ่มผู้ใช้งานอ่านไฟล์ได้	-wX
	2	ไม่อนุญาตให้ผู้ใช้งานแก้ไขไฟล์ได้	r-X
	1	ไม่อนุญาตให้ผู้ใช้งานรันไฟล์ได้	rw-
3	0	อนุญาตให้ผู้ใช้งานอื่นมีการแอ็กเซสได้เต็มที่	rwX
	4	ไม่อนุญาตให้ผู้ใช้งานอื่นอ่านไฟล์ได้	-wX
	2	ไม่อนุญาตให้ผู้ใช้งานอื่นแก้ไขไฟล์ได้	r-X
	1	ไม่อนุญาตให้ผู้ใช้งานอื่นรันไฟล์ได้	rw-

ตารางที่ 3 – ค่าของ umask

เมื่อมีการสร้างไฟล์ใหม่โดยใช้คำสั่ง open การกำหนดเพอร์มิสชันในตัวแปร mode จะถูกเปรียบเทียบกับค่าการห้ามที่กำหนดไว้ในตัวแปร umask , บิตใดที่ถูกยกเลิกใน umask ก็จะถูกยกเลิก(ถ้า

ต้องการจะแก้ไขเพิ่มเพอร์มิสชันอื่น จะใช้คำสั่ง `chmod` แก้ไขภายหลัง) เพื่อป้องกันไม่ให้มีการสร้างไฟล์ที่สามารถแอดเดสได้โดยผู้ใช้งานคนอื่นโดยอัตโนมัติ ซึ่งเป็นจุดประสงค์ของ `umask`

close

```
#include <unistd.h>
```

```
int close (int fildes);
```

จะยกเลิกการใช้งานไฟล์เดสคริปเตอร์ กับไฟล์นั้น จะทำให้ไฟล์สามารถนำไปใช้งานใหม่ได้ จะส่งค่ากลับมาเป็น 0 ถ้าทำงานได้สำเร็จ หรือ ส่งค่ากลับมาเป็น -1 ถ้ามีข้อผิดพลาดเกิดขึ้น

จำนวนไฟล์ที่โปรแกรมที่กำลังรันสามารถจะเปิดได้ในเวลาหนึ่งจะกำหนดโดยตัวแปร `OPEN_MAX` ใน `limits.h` แต่อาจจะมีข้อจำกัดอยู่บ้าง เช่น ในระบบ POSIX จะต้องกำหนดไม่ต่ำกว่า 16 เป็นต้น

ioctl

```
#include <unistd.h>
```

```
int ioctl (int fildes, int cmd , ...);
```

เป็นฟังก์ชันสำหรับควบคุมการทำงานเฉพาะอย่างของอุปกรณ์ เช่น เทอร์มินอล, ไฟล์เดสคริปเตอร์, ซอกเก็ต หรือเทปไดรฟ์ ให้ดูรายละเอียดที่ man page ของอุปกรณ์นั้น ๆ

3.6 ฟังก์ชันระดับต่ำอื่น ในการจัดการไฟล์

นอกจากนี้แล้วภาษาซี ยังมีฟังก์ชันระดับต่ำอื่น ๆ ในการจัดการไฟล์เพิ่มเติม ซึ่งมีการใช้งานไม่มากนัก

lseek

```
#include <unistd.h>
```

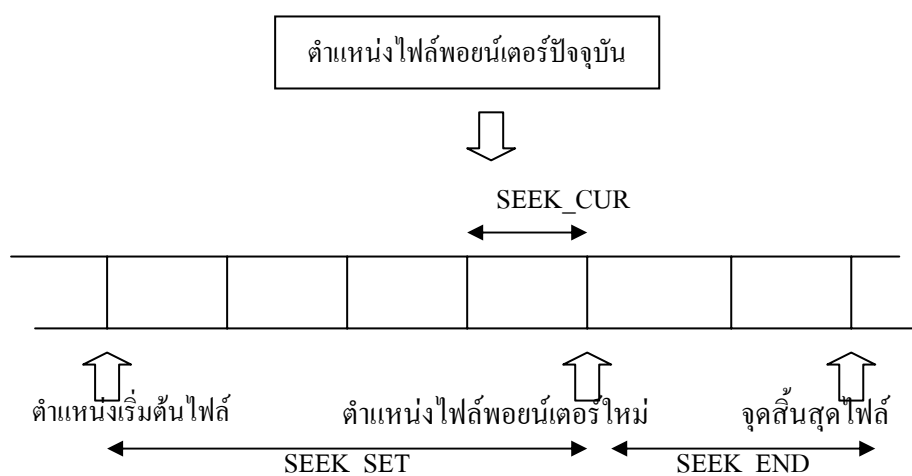
```
#include <sys/types.h>
```

```
off_t lseek (int fildes, off_t offset, int whence);
```

`lseek` จะกำหนดตำแหน่งพอยน์เตอร์ของไฟล์ที่มีไฟล์เดสคริปเตอร์ไฟล์ เพื่อกำหนดตำแหน่งที่จะอ่านหรือเขียน สามารถจะกำหนดแบบสัมบูรณ์ หรือ แบบสัมพันธ์กับตำแหน่งปัจจุบันหรือท้ายไฟล์ ค่า `offset` จะกำหนดตำแหน่งส่วน `whence` จะกำหนดลักษณะการใช้งานค่าออฟเซต ดังต่อไปนี้

SEEK_SET	offset หมายถึง ค่าสัมบูรณ์จากต้นไฟล์
SEEK_CUR	offset หมายถึงค่าสัมพันธ์กับตำแหน่งปัจจุบัน
SEEK_END	offset หมายถึงค่าสัมพันธ์กับจุดสุดท้ายของไฟล์

คำสั่ง `lseek` จะส่งค่ากลับเป็นจำนวนไบต์ ออฟเซต วัดจากจุดเริ่มต้นไฟล์ที่พอยน์เตอร์ถูกกำหนด หรือ `-1` ถ้ามีข้อผิดพลาดเกิดขึ้นระหว่างการทำงาน, สำหรับตัวแปรชนิด `offset_t` จะใช้สำหรับออฟเซต ในกรณีที่ต้องการค้นหาข้อมูล แต่บางระบบก็ไม่ใช่ใช้งาน



รูปที่ 3 – 5 ตำแหน่งต่างของ `lseek`

fstat, stat, lstat

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>

int    fstat (int fildes,    struct stat *buf);
int    stat (const char *path,    struct stat *buf);
int    lstat (const char *path,    struct stat *buf);
```

Fstat จะส่งสถานะของไฟล์ที่มีหมายเลข `fildes` ไปเก็บไว้ที่ตำแหน่ง `buf`

stat และ lstat จะส่งค่ากลับเช่นเดียวกับ `fstat` เพียงแต่ถ้าเป็น symbolic link คำสั่ง `lstat` จะให้เก็บข้อมูลเกี่ยวกับลิงค์ ขณะที่ `stat` จะให้ข้อมูลไฟล์ที่ลิงค์อ้างอิงถึง

สำหรับโครงสร้างข้อมูล stat มีรายละเอียดดังนี้

ฟิลด์ข้อมูลใน stat	คำอธิบาย
st_mode	ไฟล์เพอร์มิสชันและข้อมูลเกี่ยวกับชนิดของไฟล์
st_ino	ข้อมูลไอโนดที่เกี่ยวข้องกับไฟล์
st_dev	อุปกรณ์ที่เก็บไฟล์นี้
st_uid	Uid เจ้าของไฟล์
st_gid	Gid กลุ่มที่เจ้าของไฟล์
st_atime	เวลาในการแอ็กเซสครั้งล่าสุด
st_ctime	เวลาในการเพอร์มิสชันของไฟล์หรือข้อมูลครั้งล่าสุด
st_mtime	เวลาในการแก้ไขข้อมูลของไฟล์ครั้งล่าสุด
st_nlink	จำนวน hard link ของไฟล์

ตารางที่ 3 – 4 โครงสร้างข้อมูล stat

dup และ dup2

```
#include <unistd.h>
```

```
int dup (int fildes );
```

```
int dup2 (int fildes,      int fildes2);
```

dup จะสร้างไฟล์เดสคริปเตอร์ใหม่ เพื่อให้สามารถแอ็กเซสไฟล์เดียวกันโดยใช้ fd ไฟล์เดสคริปเตอร์ต่างกัน, อาจจะใช้งานในกรณีที่ต้องแอ็กเซส ตำแหน่งของไฟล์ที่ต่างกัน

dup2 จะทำงานเช่นเดียวกันเพียงแต่สามารถจะกำหนดค่าไฟล์เดสคริปเตอร์ใหม่ได้

3.7 ไลบรารีฟังก์ชัน

การใช้งานฟังก์ชันระดับต่ำ ก่อนข้างจะขาดความยืดหยุ่น และไม่มีประสิทธิภาพเท่าที่ควร เช่น ด้านข้อจำกัดของฮาร์ดแวร์, เทปไดรฟ์ จะมีขนาดบล็อกอย่างต่ำที่สุด 10 กิโลไบต์ ถ้าต้องการให้เล็กกว่านี้ ฮาร์ดแวร์ก็จะกำหนดให้มีขนาดพื้นที่จริง 10 กิโลไบต์ เหมือนเดิม ทำให้เกิด gap จำนวนมาก

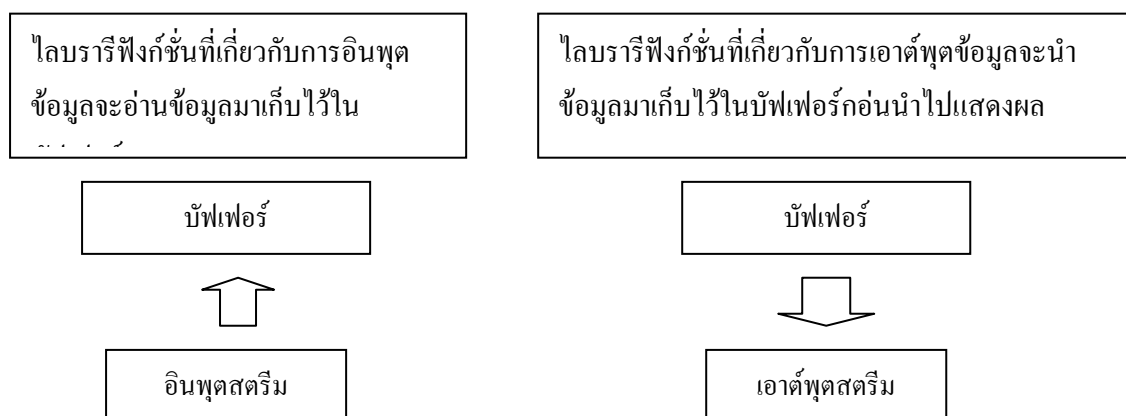
ภาษาระดับสูงที่ใช้อินเตอร์เฟซกับดิสก์หรืออุปกรณ์อื่น ๆ คือ ไลบรารี ซึ่งเป็นฟังก์ชันมาตรฐานที่สามารถจะรวมเข้าในโปรแกรม เพื่อแก้ปัญหาข้างต้น เช่น เช่น ไลบรารีเกี่ยวกับการอินพุต/เอาต์พุต ข้อมูล ทำให้สามารถจะกำหนดขนาดของบล็อกได้หลายขนาด (ไลบรารีจะไปเรียกฟังก์ชันการทำงานในระดับต่ำอีกต่อหนึ่ง)

3.8 ไลบรารีฟังก์ชันมาตรฐานสำหรับการอินพุต/เอาต์พุตข้อมูล

จะใช้ไฟล์เฮดเดอร์ `stdio.h` ติดต่อกับฟังก์ชันการทำงานในระดับต่ำ ไลบรารีจะทำให้การทำงานของอินพุต/เอาต์พุตข้อมูลทำให้ง่ายขึ้น นอกจากนี้ไลบรารียังทำหน้าที่เกี่ยวกับการกำหนดบัฟเฟอร์สำหรับการทำงานของอุปกรณ์ต่าง ๆ อีกด้วย

การใช้งานจะเหมือนกับการใช้งานฟังก์ชันระดับต่ำ จะต้องเปิดไฟล์เสียก่อนเพื่อกำหนดออฟเซตที่จะใช้อ้างอิงถึงไฟล์ เพียงแต่การเปิดไฟล์จะใช้ไฟล์สตรีมแทนไฟล์เดสคริปเตอร์ กำหนดโดยใช้โครงสร้างพอยน์เตอร์ `FILE *` เพื่อให้ฟังก์ชันอื่นสามารถจะนำไปใช้ในการทำงานกับไฟล์สตรีมได้

ปกติเมื่อมีการรันโปรแกรมจะมีไฟล์สตรีมอยู่ 3 ชนิดที่ถูกเปิดโดยอัตโนมัติคือ `stdin`, `stdout`, `stderr` (จะกำหนดไว้ใน `stdio.h`) จะเท่ากับค่าไฟล์เดสคริปเตอร์ของฟังก์ชันการทำงานในระดับต่ำ 0, 1 และ 2 ตามลำดับ



รูปที่ 3-6 การทำงานของฟังก์ชันระดับต่ำ

fopen

```
#include <stdio.h>

FILE *fopen(const char *filename, const char *mode);
```

เป็นฟังก์ชันที่สร้างขึ้นมาจาก `open` ซึ่งเป็นฟังก์ชันการทำงานระดับต่ำอีกต่อหนึ่ง ส่วนใหญ่จะใช้ในการอินพุต/เอาต์พุตข้อมูล แต่ถ้าต้องการจะแอ็กเซสไฟล์(อุปกรณ์) โดยตรง ขอแนะนำให้ใช้ `open` เนื่องจากไม่มีผลข้างเคียงจากการทำงานของไลบรารี เช่น การกำหนดบัฟเฟอร์ เป็นต้น

ฟังก์ชันจะเปิดไฟล์ชื่อ `filename` โดยกำหนดลักษณะการเปิดไฟล์ในพารามิเตอร์ `mode` ดังต่อไปนี้

“r” หรือ “rb”	เปิดไฟล์เก่าเพื่ออ่านอย่างเดียว
“w” หรือ “wb”	เปิดไฟล์ใหม่เพื่อเขียนอย่างเดียวถ้ามีไฟล์เก่าจะเขียนทับ
“a” หรือ “ab”	เปิดไฟล์เก่าเพื่อเขียนเพิ่มที่ท้ายไฟล์
“r+” หรือ “rb+” หรือ “r+b”	เปิดไฟล์เก่าเพื่ออัปเดตข้อมูล
“w+” หรือ “wb+” หรือ “w+b”	เปิดไฟล์ใหม่สำหรับอ่านและเขียน ถ้ามีไฟล์เก่าจะเขียนทับ
“a+” หรือ “ab+” หรือ “a+b”	เปิดไฟล์เก่าเพื่ออ่านและเขียนข้อมูลที่ท้ายไฟล์ ถ้าไฟล์เก่าไม่มีจะเปิดไฟล์ใหม่เพื่ออ่านและเขียน

สำหรับอักษร b จะระบุเป็นไฟล์ไบนารี อันที่จริงแล้วยูนิกซ์จะทำงานกับไฟล์เหมือนกันไม่ว่าจะเป็นเท็กซ์ไฟล์ หรือ ไบนารีไฟล์ คือจะทำงานเสมือนว่าเป็นไบนารีไฟล์ทั้งหมด เช่น การแปลอักขระควบคุมบรรทัด, สำหรับการกำหนดโหมดนั้น จะต้องกำหนดให้เป็นข้อความคือ ใช้ “r” ไม่ใช่ตัวอักษร ‘r’

ถ้าฟังก์ชันทำงานสำเร็จจะส่งค่ากลับมาเป็นพอยน์เตอร์ชี้ไปยังไฟล์นั้น ถ้าทำงานไม่สำเร็จจะส่งค่ากลับมาเป็น NULL (กำหนดไว้ใน stdio.h)

fclose

```
#include <stdio.h>

int fclose (FILE *stream);
```

จะปิดไฟล์สตรีม stream , การใช้คำสั่ง fclose เป็นสิ่งจำเป็นเนื่องจากไลบรารี stdio.h จะเก็บข้อมูลไว้ในบัฟเฟอร์ เมื่อใช้คำสั่งนี้จะทำให้โปรแกรมเขียนข้อมูลในบัฟเฟอร์กลับไปยังที่ไฟล์ อย่างไรก็ตามไฟล์จะถูกปิดโดยอัตโนมัติเมื่อสิ้นสุดการทำงานของโปรแกรม

จำนวนของไฟล์ที่สามารถเปิดได้สูงสุด จะถูกกำหนดไว้ที่ตัวแปร FOPEN_MAX ใน stdio.h

fread

```
#include <stdio.h>

size_t fread(void *ptr, size_t size, size_t nitem, FILE stream);
```

จะอ่านข้อมูลจากไฟล์สตรีม stream ไปเก็บไว้ในบัฟเฟอร์ ptr ทั้งฟังก์ชัน fread, fwrite จะจัดการข้อมูลเป็นเรกคอร์ด ขนาดของเรกคอร์ดจะกำหนดโดย size จำนวนเรกคอร์ดที่อ่านจะกำหนดโดย nitem หลังจากอ่านข้อมูลเสร็จจะส่งจำนวนเรกคอร์ดที่อ่านได้กลับ หนึ่งฟังก์ชันจะทำหน้าที่อ่านและเก็บข้อมูลไปยังบัฟเฟอร์เท่านั้น เป็นหน้าที่ของโปรแกรมเมอร์ที่จะแยกแยะข้อมูลเหล่านั้นด้วยตัวเอง

fwrite

```
#include <stdio.h>

size_t fread(const void *ptr, size_t size, size_t nitem, FILE
stream);
```

จะทำหน้าที่ตรงกันข้ามกับฟังก์ชัน fread โดยจะนำข้อมูลจากบัฟเฟอร์ ptr เขียนลงไปที่ไฟล์สตรีม stream และส่งค่ากลับเป็นเรกคอร์ดที่เขียนได้ ในกรณีเป็นจอมอนิเตอร์ stream จะเป็น stdout

fflush

```
#include <stdio.h>

int fflush (FILE *stream);
```

จะเขียนข้อมูลของไฟล์สตรีม stream ไปเก็บที่บัฟเฟอร์ทันที จะใช้ในการเก็บข้อมูลลงดิสก์เพื่อป้องกันการสูญหาย ในระหว่างการรันโปรแกรม เมื่อใช้ fclose จะเรียกฟังก์ชัน fflush โดยอัตโนมัติ

fseek

```
#include <stdio.h>

int fseek (FILE *stream, long int offset, int whence);
```

จะกำหนดตำแหน่งบนไฟล์ สำหรับการอ่านหรือเขียน offset และ whence จะใช้งานเหมือนฟังก์ชันระดับต่ำ lseek อย่างไรก็ตาม lseek จะส่งค่า offset จากต้นไฟล์กลับ แต่ fseek จะส่ง 0 ถ้าทำงานสำเร็จและ -1 ถ้าทำงานไม่สำเร็จ ขณะที่ตัวแปร errno จะระบุข้อผิดพลาดที่เกิดขึ้น

fgetc, getc, getchar

```
#include <stdio.h>

int fgetc (FILE *stream);
int gets (FILE *stream);
int getchar ();
```

fgetc จะอ่านข้อมูล 1 ไบต์จาก stream และเลื่อนพอยน์เตอร์ชี้ไปยังไบต์ถัดไป เมื่อถึงจุดสิ้นสุดของไฟล์จะส่งค่า EOF กลับ

getc	จะทำงานคล้ายกลับ fgetc เพียงแต่ทำงานในลักษณะของมาโคร
getchar	จะเท่ากับ getc(stdin) สำหรับอ่านข้อมูลหนึ่งตัวอักษรจากอุปกรณ์อินพุตมาตรฐาน

fputc, putc, putchar

```
#include <stdio.h>

int fputc (int c, FILE *stream);
int putc (int c, FILE *stream);
int putchar (int c);
```

fputc	จะเขียนข้อมูลไปยังเอาต์พุตสตรีม เมื่อถึงจุดสิ้นสุดไฟล์จะส่ง EOF กลับ
putc	จะคล้ายกลับ fputc เพียงแต่ทำงานในลักษณะของมาโคร
putchar	จะเท่ากับ putc (c, stdout) เขียนข้อมูลหนึ่งตัวอักษร ไปยังอุปกรณ์เอาต์พุตมาตรฐาน เมื่อสิ้นสุดไฟล์จะส่งค่า -1 กลับ (ไม่ใช่ EOF)

fgets, gets

```
#include <stdio.h>

char *fgets(char *s, int n, FILE *stream);
char *gets(char *s);
```

fgets	จะอ่านข้อมูลจากอินพุตสตรีม แล้วเขียนข้อมูลลงไปที่ตำแหน่งพอยน์เตอร์ s จนกระทั่งพบตัวอักษร newline หรือ รับข้อมูลไปแล้วเป็นจำนวน n - 1 ตัวอักษรหรือสิ้นสุดไฟล์ หรืออ่านข้อมูลเสร็จจะเพิ่มตัวอักษร null byte, \0 , เข้าไปที่ท้ายข้อความ รวมเป็นตัวอักษรทั้งหมด n ไบต์
-------	--

เมื่อทำงานเสร็จ fgets จะส่งค่าเป็นพอยน์เตอร์ชี้ไปยังข้อความ s ถ้าข้อมูลอินพุตสตรีมจะอยู่ที่จุดสิ้นสุดไฟล์ จะส่งค่ากลับเป็น null pointer ถ้ามีข้อผิดพลาดเกิดขึ้นระหว่างการอ่าน จะระบุข้อผิดพลาดที่เกิดขึ้นไว้ในตัวแปร errno

gets	จะอ่านข้อมูลจากอินพุตมาตรฐาน และตัดตัวอักษร newline ที่ และเพิ่มตัวอักษร null byte เช่นเดียวกับ fgets และอีกอย่างหนึ่งที่น่าสนใจก็คือ gets ไม่มีการจำกัดจำนวนตัวอักษรที่จะอ่าน ดังนั้นข้อมูลที่อยู่ในบัฟเฟอร์อาจจะถูกเขียนทับได้
------	--

3.9 ฟังก์ชันจัดการรูปแบบการอินพุตและเอาต์พุตข้อมูล

มีฟังก์ชันอยู่จำนวนหนึ่ง สำหรับควบคุมการแสดงผลข้อมูลให้เป็นไปตามที่ต้องการเช่น printf หรือ scanf สำหรับอ่านข้อมูลในรูปแบบที่ต้องการ

printf, fprintf, sprintf

```
#include <stdio.h>

int    printf (const char *format, ...);
int    fprintf (FILE *stream,    const char *format, ...);
int    sprintf(char *s,    const char *format);
```

printf	จะแสดงข้อมูลทางอุปกรณ์เอาต์พุตมาตรฐาน (กำหนดโดยพารามิเตอร์ format)
fprintf	เขียนข้อมูลไปยังสตรีมที่ต้องการ
sprintf	จะเขียนข้อมูลไปยังตัวแปรสตริง s ดังนั้นตัวแปรนี้จะต้องมีความจุมากพอสำหรับข้อมูล สำหรับข้อมูลทั่วไปจะเขียนลงไปตามปกติ ยกเว้น คอนเวอร์ชันสเปคิฟายเออร์(convension specifier) เพื่อกำหนดรูปแบบการแสดงผลข้อมูล (ส่วนใหญ่จะเริ่มด้วยตัวอักษร %)

scanf, fscanf, sscanf

```
#include <stdio.h>

int scanf (const char *format,...);
int fscanf (FILE *stream, const char *format, ...);
int sscanf (const char *s, const char *format);
```

ฟังก์ชันในตระกูล scanf จะทำงานคล้ายกับฟังก์ชันตระกูล printf เพียงแต่จะอ่านข้อมูลจากอินพุตสตรีม เก็บไว้ยังตำแหน่งหน่วยความจำที่กำหนด สำหรับรูปแบบของข้อมูลที่จะกำหนดโดยคอนเวอร์ชันสเปคิฟายเออร์ ส่วนใหญ่จะคล้ายกับที่ใช้งานในฟังก์ชันตระกูล printf

สำหรับรูปแบบของข้อมูล scanf จะประกอบด้วยตัวอักษรทั่วไปและ คอนเวอร์ชันสเปคิฟายเออร์ แตกต่างกับที่ คอนเวอร์ชันสเปคิฟายเออร์ จะวางไว้ที่ส่วนอินพุตของข้อมูล

ฟังก์ชัน scanf จะส่งค่ากลับเป็นจำนวนเรกคอร์ดที่อ่านได้ ถ้ามีการอ่านจนถึงจุดสิ้นสุดไฟล์จะส่งค่า EOF กลับ ถ้ามี error เกิดขึ้นในระหว่างการอ่านข้อมูลจากอินพุตสตรีม จะกำหนด error flag เพื่อแจ้งการเกิดความผิดพลาด พร้อมกับระบุชนิดของความผิดพลาดไว้ที่ตัวแปร errno

3.10 การจัดการกับไฟล์และไดเรกทอรี

ไลบรารีฟังก์ชันสำหรับทำงานกับไฟล์และไดเรกทอรี จะมีดังนี้

chmod

จะเปลี่ยนเพอร์มิสชันในการแอ็กเซสไฟล์ หรือ ไดเรกทอรี

```
#include <sys/stat.h>
```

```
int      chmod (const char *path,  mode_t mode);
```

ไฟล์ที่ต้องการเปลี่ยนเพอร์มิสชันจะกำหนดโดย path โดยใช้เพอร์มิสชัน mode เช่นเดียวกับ open

unlink, link, symlink

```
#include <unistd.h>
```

```
int      unlink (const char * path);
```

```
int      link (const char *path1,   const char *path2);
```

```
int      symlink(const char *path1,const char *path2);
```

unlink ใช้สำหรับลบลิงก์ไฟล์ที่ต้องการ จะส่งค่ากลับมาเป็น 0 ถ้าทำงานสำเร็จ หรือ -1 ถ้ามีข้อผิดพลาดเกิดขึ้น อย่างไรก็ตามการจะลบได้ต้องมีเพอร์มิสชันที่เพียงพอด้วย โปรแกรม rm ก็ใช้ฟังก์ชันการทำงานนี้

link จะสร้างลิงก์ของไฟล์ path1 ชื่อ path2 สำหรับการสร้างซิมโบริก ลิงก์จะใช้ฟังก์ชัน symlink ใช้งานในทำนองเดียวกัน ในเชลล์จะใช้คำสั่ง ln สร้างลิงก์ที่ต้องการ ในขณะที่การเขียนโปรแกรมจะใช้ฟังก์ชัน link แทน

ลิงก์ (Link) จะมีอยู่ 2 ชนิด คือฮาร์ดลิงก์(hard link) เป็นชื่อไฟล์ในตารางไอโนด ที่ชี้ไปยังตำแหน่งข้อมูลเดียวกันบนฮาร์ดดิสก์ และซิมโบริก ลิงก์(symbolic link) เป็นเท็กซ์ไฟล์เก็บข้อความระบุตำแหน่งของไฟล์(ไม่อยู่ในตารางไอโนด)

mkdir, rmdir

```
#include <sys/type.h>
```

```
int      mkdir (const char *path,  mode_t mode);
```

ฟังก์ชัน mkdir จะสร้างไดเรกทอรี path ที่กำหนด โดยใช้เพอร์มิสชัน mode (สามารถใช้งานออฟชั่น O_CREAT ของฟังก์ชัน open และได้รับผลจากคำสั่ง umask ด้วยเช่นกัน)


```
#include <unistd.h>
```

```
int      rmdir (const char *path);
```

ฟังก์ชัน rmdir จะลบไดเรกทอรี path ที่ไม่มีข้อมูลในไดเรกทอรี ออกจากระบบ

chdir, getwd

ฟังก์ชันระดับต่ำ chdir จะเปลี่ยนไปยังไดเรกทอรีที่ต้องการ(เหมือนกับคำสั่ง cd บนเชลล์)

```
#include <unistd.h >
```

```
int      chdir(const char *path);
```

ส่วนไลบรารีฟังก์ชัน getwd จะส่งค่ากลับเป็นไดเรกทอรีที่กำลังทำงานอยู่ปัจจุบัน

```
#include <unistd.h>
```

```
char     *getwd(char *buf,      size_t size);
```

โดยจะเขียนชื่อของไดเรกทอรีปัจจุบันในบัฟเฟอร์ buf หรือจะส่งค่ากลับมาเป็น null ถ้ามี error เกิดขึ้น เช่น ชื่อมีความยาวมากกว่าความจุบัฟเฟอร์ ซึ่งกำหนดโดยพารามิเตอร์ size

3.11 ตรวจสอบไดเรกทอรี

ไลบรารีฟังก์ชันได้ถูกพัฒนาขึ้นมาเพื่อให้ทำงานกับไดเรกทอรีทำได้ง่ายขึ้น ฟังก์ชันกำหนดไว้ที่ไฟล์ dirent.h โดยชื่อโครงสร้างข้อมูลชื่อ DIR สำหรับจัดการกับไดเรกทอรี พอยน์เตอร์ชี้ไปยังโครงสร้างนี้เรียกว่า directory stream pointer (DIR) ฟังก์ชันที่ทำงานกับไดเรกทอรีจะมีอยู่หลายอย่างดังต่อไปนี้

opendir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
DIR      *opendir (const char *name);
```

ฟังก์ชัน opendir จะเปิด(เริ่ม)การทำงานกับไดเรกทอรี ถ้าทำสำเร็จจะส่งค่ากลับเป็นพอยน์เตอร์ชี้ไปยังโครงสร้างข้อมูล DIR เพื่ออ่านข้อมูลในไดเรกทอรี ถ้าทำงานไม่สำเร็จจะส่งค่ากลับมาเป็น null pointer

readdir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
struct dirent *readdir(DIR *dirp);
```

จะส่งค่ากลับเป็นพอยน์เตอร์ชี้ไปยังข้อมูลถัดไปในไดเรกทอรีสตรีม dirp เมื่อถึงจุดสิ้นสุดข้อมูล จะส่งค่ากลับเป็น null สำหรับโครงสร้างข้อมูล dirent จะเก็บข้อมูลของไฟล์ในไดเรกทอรีที่สำคัญคือ

ino_t	d_ino	inode ของไฟล์
char	d_name[]	ชื่อไฟล์

telldir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
long int telldir (DIR *dirp);
```

จะส่งค่ากลับเป็นจำนวนตัวเลขแสดงตำแหน่งปัจจุบันในไดเรกทอรีสตรีม จะใช้งานก่อนที่จะใช้

seekdir

seekdir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
void seekdir(DIR *dirp, long int loc);
```

จะกำหนดตำแหน่งของพอยน์เตอร์ในไดเรกทอรีสตรีม dirp ค่า loc จะกำหนดตำแหน่งที่ต้องการ ถ้าต้องการรีเซ็ตพอยน์เตอร์ให้อยู่ในตำแหน่งปัจจุบัน ควรใช้ค่าส่งกลับจากฟังก์ชัน telldir

closedir

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
int closedir (DIR *dirp);
```

จะปิดการทำงานของไดเรกทอรีสตรีม รีซอร์สต่าง ๆ ที่ใช้สำหรับไดเรกทอรีสตรีม สามารถจะนำไปใช้กลับงานอื่น ๆ ได้ จะส่งค่ากลับเป็น 0 ถ้าทำงานสำเร็จ หรือ -1 ถ้าทำงานไม่สำเร็จ

3.12 การระบุ error ของฟังก์ชัน

หลายฟังก์ชันในไลบรารี stdio จะส่งค่าความผิดพลาดกลับเป็นค่าที่เป็นไปไม่ได้ เช่น จำนวนที่มากจนเกินไป หรือ null pointer หรือ EOF ส่วนการระบุชนิดของความผิดพลาดที่เกิดขึ้นจะเก็บไว้ในตัวแปร errno ซึ่งเป็นตัวแปรที่ใช้งานกันได้ทั่วโปรแกรม (External variable)

```
#include <errno.h>
```

```
extern int errno;
```

นอกจากนี้ยังมีฟังก์ชันที่ใช้ค้นหาว่า มีข้อผิดพลาดเกิดขึ้นหรือไม่ หรือ ถึงจุดสิ้นสุดไฟล์หรือยัง โดยใช้ฟังก์ชันการทำงานต่อไปนี้

```
#include <stdio.h>

int    perror(FILE *stream);

int    feof (FILE *stream);

void    clearerr (FILE *stream);
```

ferror	จะทดสอบว่ามีการกำหนดค่า errno หรือเปล่า ส่งค่ากลับเป็นตัวเลขไม่เท่ากับ 0 ถ้ามีการกำหนด หรือจะส่งค่ากลับเป็น 0 ถ้าไม่มีการกำหนดค่าให้กับ errno
feof	จะทดสอบว่าถึงจุดสิ้นสุดไฟล์หรือยัง ถ้ายังไม่ถึงจุดสิ้นสุดไฟล์จะส่งค่าที่ไม่เท่ากับ 0 กลับแต่ถ้าถึงจุดสิ้นสุดไฟล์จะส่งค่า 0
clearerr	จะลบข้อมูล EOF หรือข้อมูลใน errno ที่ stream ซ้ำอยู่ ไม่มีการส่งค่ากลับเป็นการเคลียร์ตัวแปรเท่านั้น

การใช้งานตัวแปร errno เป็นวิธีการทั่ว ๆ ไปสำหรับไลบรารี ดังนั้นถ้าเป็นการทำงานที่อาจจะมีการผิดพลาดเกิดขึ้น ควรจะตรวจสอบตัวแปรนี้ทันที เนื่องจากอาจถูกเขียนทับโดยฟังก์ชันการทำงานอื่น ค่าที่กำหนดให้กับค่าของตัวแปร errno จะกำหนดไว้ในไฟล์ errno.h ดังต่อไปนี้

EBUSY	อุปกรณ์หรือ รีซอร์สไม่สามารถทำงานตามที่ต้องการ
EEXIST	มีไฟล์อยู่
ENODEV	ไม่มีอุปกรณ์
EISDIR	เป็นไดเรกทอรี
ENOTDIR	ไม่ใช่ไดเรกทอรี
EINVAL	ใช้งานอาร์กิวเมนต์ไม่ถูกต้อง
EMFILE	เปิดไฟล์มากเกินไป
EINTR	เกิดการอินเตอร์รัพในระหว่างการทำงานของฟังก์ชันระดับต่ำ
EIO	เกิดข้อผิดพลาดในกระบวนการ อินพุต/เอาต์พุต
EPERM	ไม่ได้รับเพอร์มิสชันสำหรับการทำงานดังกล่าว
ENOENT	ไม่มีไฟล์หรือไดเรกทอรี

มีฟังก์ชันบางอย่างในการทำงานกับความผิดพลาดคือ `strerror` และ `perror`

```
#include <string.h>
```

```
char *strerror (int errnum);
```

ฟังก์ชัน `strerror` จะแม็พหมายเลขของความผิดพลาด เป็นคำอธิบายที่เข้าใจได้ง่าย ใช้สำหรับบันทึกใน ไฟล์ `log`

```
#include <stdio.h>
```

```
void perror (const char *s);
```

ฟังก์ชัน `perror` จะแม็พความผิดพลาดเป็นคำอธิบายและแสดงออกทาง `standard error stream` และสามารถกำหนดให้นำหน้าตัวอักษรในสตริง `s` ตามด้วยเครื่องหมายโคลอนและช่องว่าง(ไม่มีก็ได้, `null`)